

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	CSE-AI
Programme:	B.E/B.TECH
Course Code & Course Name	CSA1209 - COMPUTER ARCHITECTURE
Academic Year / Batch	2025
Faculty Name	D. SANTHAKUMAR
Assignment Title	Datapath, Superscalar Dispatch and Virtual-Memory Design for an Autonomous Drone Flight Controller
Date of Issue	01/09/2026
Date of Submission	03/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO4 – Apply computer architecture concepts to design and evaluate processor and memory systems
Bloom's Taxonomy Level	L5 – Evaluate
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Autonomous drones, real-time embedded systems, robotics, aerospace applications, and safety-critical intelligent systems

B. Assignment Problem / Challenge

An autonomous drone flight controller must process sensor data, perform navigation and control calculations, and generate motor-control instructions in real time. The processor architecture must provide efficient instruction execution, parallel processing and fast memory access to satisfy the

computational requirements of autonomous flight.

In this assignment, students design and analyze a processor architecture consisting of a datapath, superscalar dispatch mechanism and virtual-memory subsystem. The design includes instruction execution through the datapath, dual-issue instruction dispatch for improving instruction-level parallelism, and paging with TLB-based address translation for efficient memory management.

The assignment requires students to analyze instruction dependencies, pipeline behavior, dispatch performance, virtual-address translation and TLB misses. Suitable architectural parameters and constraints are selected and the proposed design is implemented or simulated using appropriate tools such as Logisim Evolution, Python, MATLAB or equivalent simulation environments.

The performance of the proposed architecture is evaluated using suitable test cases, simulation outputs and quantitative measures such as execution cycles, instruction throughput, dispatch speed-up, TLB hit/miss behavior and address-translation overhead. Alternative design choices are analyzed based on performance, complexity, reliability, power efficiency and suitability for real-time autonomous drone operation.

The final solution is validated through simulation results and engineering analysis, with appropriate consideration of safety, reliability, energy efficiency, societal impact and responsible use of autonomous drone technology.

C. Problem Statement

An autonomous drone flight controller requires a high-performance processor to execute sensor-processing, navigation and flight-control instructions with minimum delay. The proposed system shall integrate a processor datapath, dual-issue superscalar dispatch mechanism and virtual-memory subsystem to improve instruction throughput and memory-access efficiency.

The design shall include suitable components for instruction execution, arithmetic operations, instruction dispatch, pipeline processing and memory management. The superscalar unit shall support the parallel dispatch of independent instructions while handling instruction dependencies and pipeline hazards. The virtual-memory subsystem shall implement paging, page-table mapping and TLB-based address translation, including TLB-miss and page-fault handling.

The proposed architecture shall be implemented or simulated using appropriate tools such as Logisim Evolution, Python, MATLAB or equivalent tools. The design shall be tested using suitable instruction sequences and memory-address traces. Performance shall be evaluated using execution cycles, instruction throughput, superscalar speed-up, TLB hit/miss behavior and address-translation overhead.

The final design shall demonstrate efficient, reliable and predictable processing suitable for the real-time requirements of an autonomous drone flight controller, while considering hardware complexity, power efficiency and safety.

D. Requirements and Constraints

The proposed autonomous drone flight-controller architecture shall satisfy the following requirements and constraints:

Functional Requirements

- Implement a suitable processor datapath for instruction execution.
- Support arithmetic, logical, memory and control operations.
- Support dual-issue superscalar instruction dispatch.
- Identify and handle instruction dependencies and pipeline hazards.
- Implement paging-based virtual-memory management.
- Provide TLB-based virtual-to-physical address translation.
- Handle TLB misses and page faults.

Performance Requirements

- Improve instruction throughput using superscalar execution.
- Reduce unnecessary pipeline stalls.
- Minimize instruction execution cycles.
- Achieve efficient TLB hit performance.
- Reduce virtual-address translation overhead.
- Provide predictable processing for real-time flight-control operations.

Technical Constraints

- 32-bit processor architecture.
- 32 general-purpose registers.

- Dual-issue instruction dispatch.
- 4 KB page size.
- 16-entry TLB.
- 64 MB main memory.
- Suitable instruction and memory-address traces for simulation.

Cost and Time

- Use freely available or institutionally accessible simulation tools.
- Complete the design, implementation and testing within the assigned submission period.
- Minimize unnecessary hardware complexity.

Resources

- Logisim Evolution for datapath and processor design.
- Python or MATLAB for performance and memory simulation.
- Suitable instruction sequences and virtual-address traces.
- Computer system capable of running the selected simulation tools.

Safety

- Ensure correct execution of flight-control instructions.
- Avoid unpredictable delays during critical processing.
- Validate the architecture before considering practical autonomous-flight applications.

Reliability

- Correct handling of instruction dependencies.
- Reliable pipeline operation.
- Correct virtual-to-physical address translation.
- Proper handling of TLB misses and page faults.

Sustainability and Power/Energy

- Efficient utilization of processor resources.
- Balance performance improvement with hardware complexity and energy consumption.
- Avoid unnecessary instruction execution and memory operations.

Environmental and Ethical Considerations

- Consider responsible use of autonomous drone technology.
- Minimize unnecessary energy consumption.
- Consider the societal impact and safe deployment of autonomous systems.

Security/Privacy

- Protect sensitive sensor and flight-related data.
- Consider secure access to memory and control information where applicable.

User Requirements

- Real-time response.
- Reliable flight-control processing.
- Efficient execution of sensor and navigation workloads.
- Stable and predictable system performance.

These are the relevant constraints for this particular Computer Organization and Architecture problem; unrelated categories from the generic template are not required. The uploaded format also specifies that only constraints relevant to the particular course/problem need to be considered.

1. Problem Understanding and Formulation

The assignment focuses on designing an efficient processor architecture for an autonomous drone flight controller. The processor must handle sensor-data processing, navigation calculations, flight-control instructions and memory operations with low execution delay.

The expected outcomes are:

- Efficient instruction execution through a suitable datapath.
- Improved instruction throughput using dual-issue superscalar dispatch.
- Proper handling of instruction dependencies and pipeline hazards.
- Efficient virtual-to-physical address translation.
- Effective handling of TLB misses and page faults.
- Improved processor and memory performance for real-time drone operations.

The available information includes the processor configuration, instruction sequences, memory-address traces, datapath components, superscalar issue width, page size, TLB

configuration and main-memory parameters.

Assumptions

- 32-bit processor architecture.
- 32 general-purpose registers.
- Dual-issue superscalar processor.
- 4 KB page size.
- 16-entry TLB.
- 64 MB main memory.
- Five-stage pipeline consisting of Fetch, Decode/Dispatch, Execute, Memory and Write Back.
- Suitable instruction and memory traces are used for simulation.

Constraints

The design must maintain correct instruction execution, handle data dependencies, minimize pipeline stalls, provide efficient address translation and maintain predictable processing performance suitable for autonomous drone control.

2. Application of Course Knowledge

The assignment applies the following Computer Organization and Architecture concepts:

Principles

- Processor organization
- Datapath operation
- Instruction execution
- Pipelining
- Instruction-level parallelism
- Memory hierarchy

Theories

- Pipeline execution
- Superscalar architecture
- Instruction dependency and hazard handling
- Virtual-memory organization
- Paging and TLB operation

Mathematical Models

- Instruction throughput
- Clock-cycle calculation
- Speed-up calculation
- TLB hit ratio
- Memory-access time
- Effective memory access time

Algorithms

- Dual-issue instruction dispatch
- Dependency detection
- Pipeline scheduling
- Page-table address translation
- TLB lookup
- TLB-miss handling
- Page-fault handling

Engineering Concepts

- Processor datapath design
- Functional-unit utilization
- Performance optimization
- Hardware complexity versus performance
- Real-time processing

Necessary calculations include execution cycles, instruction throughput, superscalar speed-up, TLB hit/miss rate and address-translation overhead.

3. Solution / Design / Methodology

Datapath Design

The proposed datapath consists of:

Program Counter → Instruction Memory → Register File → ALU → Data Memory → Register File

The design includes:

- Program Counter
- Instruction Memory
- Register File
- ALU
- Control Unit
- Multiplexers
- Data Memory
- Pipeline Registers

Superscalar Dispatch Design

A dual-issue pipeline is used:

Fetch → Decode/Dispatch → Execute → Memory → Write Back

Two independent instructions can be dispatched in the same clock cycle. Dependency checking is performed before issuing instructions to functional units.

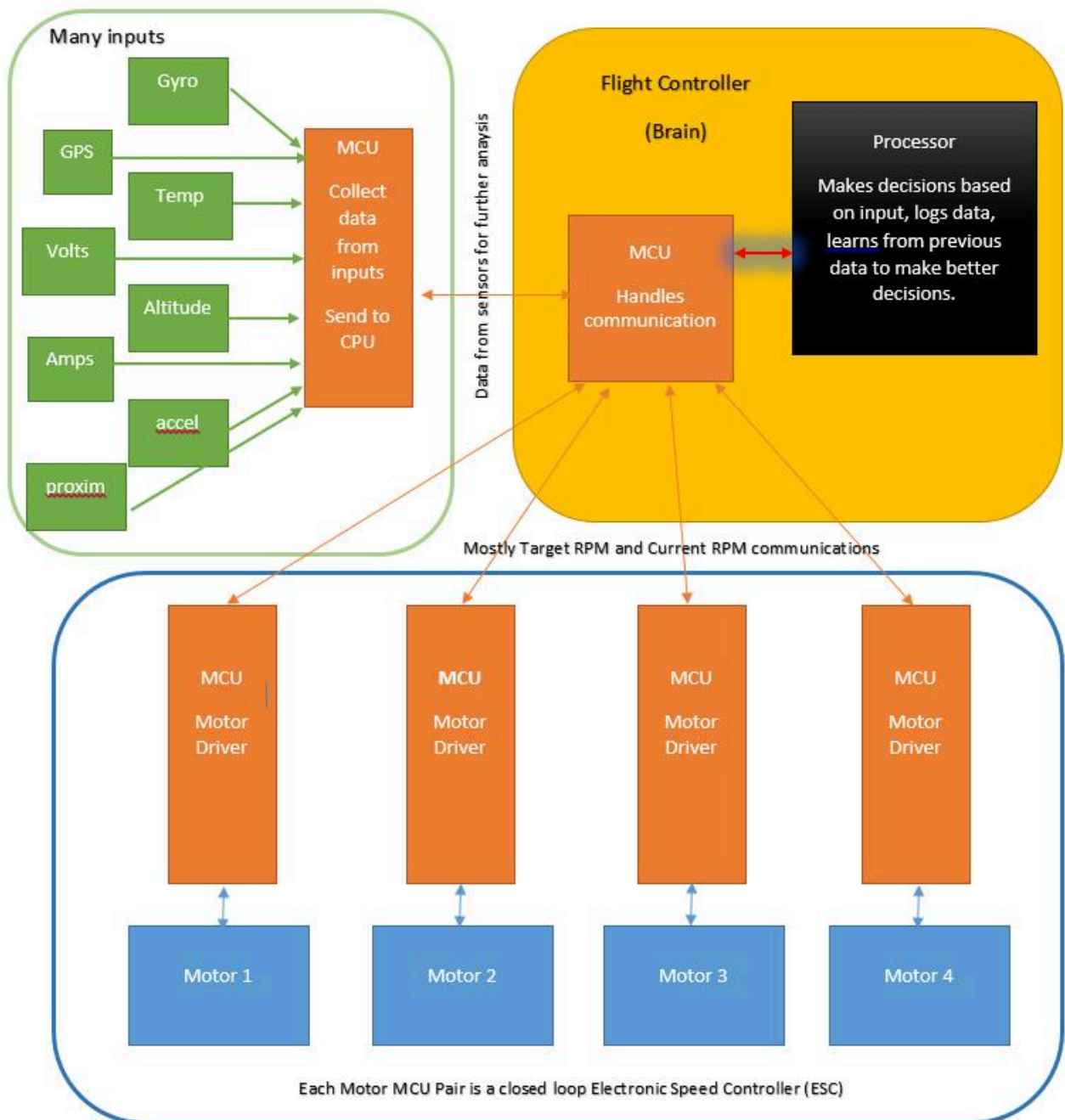
Virtual-Memory Design

The proposed memory-management scheme uses paging and TLB-based translation:

Virtual Address → TLB → Page Table → Physical Frame → Main Memory

The virtual address is divided into a page number and page offset. The TLB stores frequently used page-to-frame mappings. A TLB hit provides faster translation, while a TLB miss results in page-table lookup. An invalid page-table entry generates a page fault.

SYSTEM ARCHITECTURE:



4. Use of Modern Tools

The proposed Autonomous Drone Flight Controller architecture was designed and simulated using modern engineering and computing tools. Logisim was used to design and simulate the

processor datapath, including the PC, instruction memory, registers, ALU, multiplexer and data memory. Google Colab with Python was used to implement and simulate the dual-issue superscalar dispatch mechanism and the virtual-memory system with TLB-based address translation. The simulation outputs were used to observe instruction execution, dispatch cycles, TLB hits, TLB misses and physical-address translation.

Tools Used:

- Logisim – Datapath design and simulation
- Google Colab – Program execution and simulation environment
- Python – Superscalar dispatch and virtual-memory/TLB simulation

5. Results and Validation

The proposed datapath successfully supports the required instruction execution operations of the autonomous drone flight controller. The processor executes arithmetic, logical and memory instructions through the designed datapath with proper control and data flow.

The dual-issue superscalar mechanism improves instruction throughput by allowing two independent instructions to be dispatched in the same clock cycle. The execution performance is evaluated by comparing the number of clock cycles required by single-issue and dual-issue execution.

The virtual-memory subsystem successfully performs virtual-to-physical address translation using the TLB and page table. TLB hits provide faster address translation, while TLB misses initiate page-table lookup. Page-fault conditions are also considered as edge cases during testing.

The architecture is validated using suitable instruction sequences, memory-address traces and simulation outputs. The obtained results demonstrate correct datapath operation, improved instruction throughput and proper handling of TLB hits, TLB misses and page faults.

The simulation screenshots and execution outputs provide evidence that the proposed architecture satisfies the functional and performance requirements of the autonomous drone flight controller



```
# Superscalar Dispatch Simulator

instructions = [
    ("ADD", "R1", "R2"),
    ("SUB", "R3", "R4"),
    ("MUL", "R5", "R1"),
    ("ADD", "R6", "R3"),
    ("SUB", "R7", "R5")
]

# Single Issue
single_cycles = len(instructions)

# Dual Issue
cycles = 0
i = 0

while i < len(instructions):
    cycles += 1
    if i + 1 < len(instructions):
        # Check dependency
        if instructions[i][1] != instructions[i+1][2]:
            i += 2
        else:
            i += 1
    else:
        i += 1

print("Single Issue Cycles:", single_cycles)
print("Dual Issue Cycles:", cycles)
print("Speed-up:", round(single_cycles / cycles, 2))
```

```
... Single Issue Cycles: 5
Dual Issue Cycles: 3
Speed-up: 1.67
```

Virtual Memory + TLB output screenshot

```
# Virtual Memory and TLB Simulator

page_size = 4
tlb = {0: 5, 1: 2, 3: 7}
page_table = {0: 5, 1: 2, 2: 8, 3: 7}

addresses = [2, 5, 10, 14, 20]

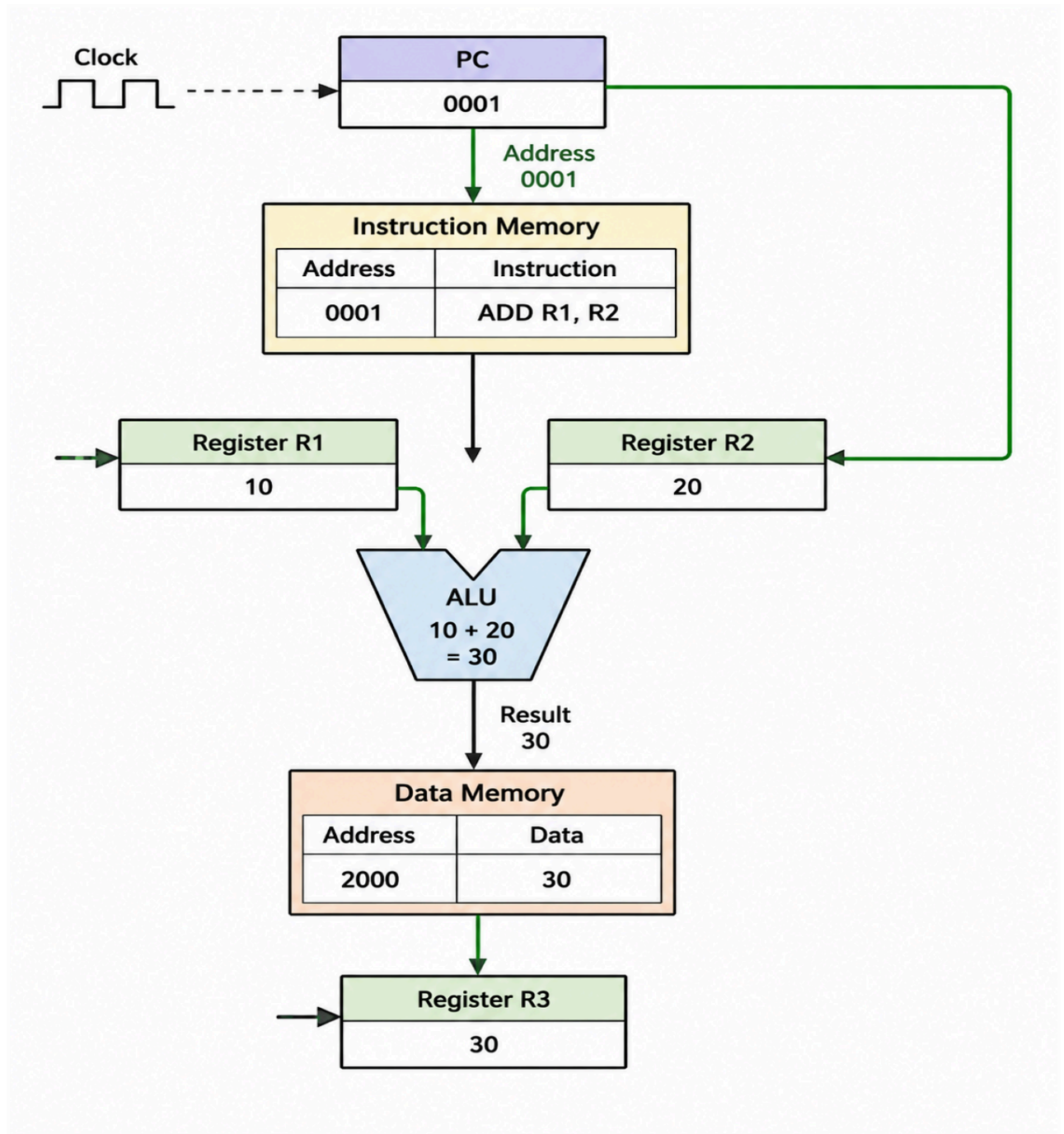
for addr in addresses:
    page = addr // page_size
    offset = addr % page_size

    if page in tlb:
        frame = tlb[page]
        result = "TLB Hit"
    elif page in page_table:
        frame = page_table[page]
        result = "TLB Miss"
    else:
        print("Address:", addr, "Page Fault")
        continue

    physical = frame * page_size + offset
    print("Address:", addr, "| Page:", page,
          "|", result, "| Physical:", physical)

... Address: 2 | Page: 0 | TLB Hit | Physical: 22
Address: 5 | Page: 1 | TLB Hit | Physical: 9
Address: 10 | Page: 2 | TLB Miss | Physical: 34
Address: 14 | Page: 3 | TLB Hit | Physical: 30
Address: 20 Page Fault
```

Datapath – Logisim



6. Analysis and Engineering Decision

The single-issue and dual-issue architectures are compared based on execution cycles and instruction throughput. The dual-issue architecture provides improved throughput when sufficient independent instructions are available.

Instruction dependencies can reduce the theoretical performance improvement by introducing pipeline stalls. Proper dependency detection prevents incorrect parallel execution.

The TLB improves memory-access performance by providing fast translation for frequently accessed pages. TLB misses introduce additional page-table lookup overhead, while page faults produce significantly higher memory-management delays.

The dual-issue superscalar architecture with TLB-based virtual memory is selected because it provides a suitable balance between performance, memory efficiency, reliability and hardware complexity.

7. Broader Considerations

Sustainability

Efficient instruction execution and memory access can reduce unnecessary processor activity and energy consumption.

Environment

Energy-efficient processor operation can reduce the overall power requirement of autonomous drone systems.

Society

Autonomous drones can support applications such as logistics, infrastructure inspection, disaster management and environmental monitoring.

Safety

Reliable and predictable processor execution is essential for flight-control operations and safe autonomous behavior.

Ethics

Autonomous drone systems should be designed and deployed responsibly, with consideration for privacy, safety and appropriate usage.

Economics

Performance improvements should be balanced against additional hardware complexity and implementation cost.

Accessibility

Efficient and modular architectural designs can support broader development and educational use of autonomous-system technologies.

Professional Responsibility

The architecture must be properly simulated, tested, documented and validated before being considered for practical deployment.

8. Conclusion

The proposed design integrates a processor datapath, dual-issue superscalar dispatch mechanism and paging-based virtual-memory subsystem for an autonomous drone flight controller.

The datapath provides the basic instruction-execution structure, while superscalar dispatch improves instruction throughput by allowing independent instructions to execute in parallel. The virtual-memory subsystem provides efficient address translation through TLB and page-table mechanisms.

The design demonstrates the application of Computer Organization and Architecture concepts to a real-time autonomous-system problem. Further improvements can include out-of-order execution, branch prediction, larger TLB capacity, cache optimization and wider superscalar issue.

9. Student Reflection

The assignment provides practical understanding of datapath organization, pipelining, superscalar execution and virtual-memory management. The design process connects theoretical processor concepts with a real-world autonomous drone application.

The implementation improves understanding of instruction dependencies, pipeline hazards, instruction-level parallelism, TLB operation and memory-address translation.

With additional resources, the design can be improved by incorporating cache-memory optimization, branch prediction, out-of-order execution and more advanced superscalar scheduling to achieve higher performance.

10. References

1. William Stallings, *Computer Organization and Architecture*, Pearson.
2. David A. Patterson and John L. Hennessy, *Computer Organization and Design*, Morgan Kaufmann.
3. Carl Hamacher, Safwat Zaky and Zvonko Vranesic, *Computer Organization*.
4. Logisim Evolution Documentation.
5. Python Documentation.
6. MATLAB Documentation.
7. Relevant processor architecture and virtual-memory references used during implementation.